Обзор литературы к предрелизной подготовке системы MarLine

Овечкин Андрей

Группа: *25.Б72-мм*
Кафедра: *системного программирования*
Научный руководитель: *Гориховский Вячеслав Игоревич*
Консультант: *Дмитриевцев Алексей Сергеевич, младший инженер по разработке ПО, ООО "КНС ГРУПП"*

1. Введение

Цель практики – довести систему MarLine до состояния, при котором её можно выпускать без ручной проверки: покрыть конвейер интеграционными тестами, устранить найденные дефекты и привести репозиторий к публикуемому виду. Прежде чем трогать код, нужно понять, на каких идеях система стоит, где она повторяет уже опубликованные решения, а где выходит за их рамки. Без этого не получится ни осмысленно чинить хешеры и индексы, ни объяснить, в чём новизна работы. Ниже рассмотрены публикации, на которые MarLine опирается архитектурно, а в конце сделаны выводы о том, что из этого следует для предрелизной подготовки.

2. Разбиение на чанки

Дедупликация начинается с разбиения данных на блоки. Если резать по фиксированному размеру, любая вставка в начало файла сдвигает все границы, и неизменённая часть перестаёт совпадать с уже сохранённой копией. Поэтому в современных системах используют разбиение, зависящее от содержимого: граница ставится там, где свертка скользящего окна укладывается в заданное условие, и положение границы определяется самими данными, а не порядковым номером байта.

Полиномиальное хэширование на каждом байте дорого, поэтому FastCDC [1] заменил её Gear-хешем, который обновляется за один байт, и добавил три приёма, избавляющих от лишней работы: упрощённую проверку условия разреза, пропуск точек ближе минимального размера чанка и нормализацию распределения размеров. Это дало примерно десятикратное ускорение при том же качестве. Версия 2020-го года [2] довела проект до практики: приёмов стало пять, включая прокрутку окна на два байта, ускорение выросло до 3–12×, а на действующей системе Destor (открытая платформа для экспериментов по дедупликации) интеграция FastCDC ускорила сквозную запись в 1.2–3.0×. Для нас важно, что вторая статья проверяет алгоритм не в вакууме, а внутри настоящего хранилища, – она задаёт стандарт не только на сам чанкер, но и на то, как его оценивать.

MarLine использует разбиение именно этой версии, совмещая его с отдельными приёмами из статьи 2016 года. При предрелизной работе с `fastcdc.rs` это означает, что таблицы Gear-констант и маски выглядят взятыми из ниоткуда, но у каждой из них есть обоснование в одной из двух статей, и при ревью кода удобно возвращаться к первоисточнику, а не восстанавливать смысл по догадкам.

3. Определение сходства

Дедупликация распознаёт лишь идентичные блоки. Данные же резервных копий часто похожи, но не совпадают: изменилась пара функций, файл пересобран с другим набором флагов. Чтобы такие блоки сжимать, их нужно сначала найти, а поиск похожих пар среди миллионов чанков – самая дорогая часть работы.

Начиналась эта линия с системы Ароновича [3]. Её метод сбора признаков, известный как N-Transform, проходит хешем скользящего окна по всему блоку и оставляет несколько наименьших значений в качестве признаков. Авторы заметили, что распределение сверток неравномерно, и поправили отбор: берутся четыре минимума, после чего поиск продолжается со сдвигом. Значение этой работы для нас в том, что все последующие публикации измеряют себя относительно N-Transform, – это точка отсчёта, и её же мы видим в коде как хешер сходства.

Дальше линия быстро расходится. ODESS [4] показал, что просчитывать свертку на каждом окне не обязательно: устойчивый признак можно получить из выборки, зависящей от содержимого, и применить преобразования только к ней. Признаки стали генерироваться в 5.4× быстрее, чем у Finesse, и в 26.9× быстрее, чем у N-Transform, при том же коэффициенте сжатия. Finesse, промежуточный вариант, ускорял N-Transform ценой точности и в итоге ниже фигурирует только как базовое сравнение.

Palantir [5] предложил строить не один набор грубых признаков, а иерархию: широкий слой находит как можно больше вариантов, узкий отсеивает лишние. Сжатие бэкапа выросло на 7.3% относительно N-Transform и ODESS и на 26.5% относительно Finesse при потере скорости не более 7.7%; дополнительные суперпризнаки авторы компенсировали тем, что похожие данные в бэкапах приходят группами. Наконец, SpeedSketch [6] объединил поиск и кодирование в один конвейер: скетч в стиле фильтра Блума не только ищет похожие блоки, но и сразу отбрасывает те, которые всё равно не сожмутся. Генерация скетча в 1.59× быстрее, чем у ODESS, кодирование в 1.92× быстрее Gdelta.

Для MarLine эта часть литературы важна ещё больше. Во-первых, система содержит сразу две ветви сходства (ODESS-хешер в `marline_sketcher` и иерархические суперпризнаки в `marline_palantir`), то есть внутри одного проекта сосуществуют две линии из литературы, и их можно измерять на общих данных. Во-вторых, именно здесь находится большинство найденных дефектов: недетерминированные коэффициенты, из-за которых признаки двух экземпляров хешера не совпадают, и ошибки в семантике индекса, из-за которых повторные признаки искажают подсчет пересечений.

4. Дельта-кодирование

Найденную пару похожих блоков нужно записать компактно. Здесь состояние поля удобно описать относительно Gdelta [7]: скорость ему даёт Gear-хеш на мелких словах и массивная индексация, степень сжатия – пакетная обработка остатка после сопоставления слов. По замерам авторов, на шести наборах данных кодирование и декодирование в 2–4× быстрее Xdelta и Zdelta при росте коэффициента сжатия на 10–120%.

Интересно, что в работах по определению сходства [4, 5, 6] Gdelta выступает только как база для сравнения и даже не цитируется. Это говорит о том, что задача дельта-кодирования считается решённой, а исследовательский интерес лежит в поиске похожих пар.

5. Платформа

Все перечисленные алгоритмы в MarLine живут не сами по себе, а внутри ChunkFS [8] – in-memory файловой системы, где алгоритмы разбиения, хеширования и хранения подменяются через трейты `Chunker` и `Scrub`. Здесь проходит граница между кодом: всё, что относится к разбиению и хранению, система получает от платформы, а собственные алгоритмы сходства и сжатия подключаются как скрабберы. Для предрелизной подготовки это означает, что значительная часть конвейера – внешняя зависимость, и её состояние нужно учитывать отдельно от состояния собственного кода.

6. Анализ

Сведём рассмотренные методы определения сходства в таблицу по двум метрикам: скорости генерации признаков и качеству сжатия. Приведены только значения, указанные в источниках; там, где прямого числа нет, стоит прочерк.

Метод	Скорость признаков	Качество сжатия
N-Transform	$1\times$ (эталон)	$1\times$ (эталон)
Finesse	—	—
ODESS	в $26.9\times$ быстрее N-Transform	как у N-Transform
Palantir	медленнее N-Transform на $\leq 7.7\%$	на 7.3% больше, чем у N-Transform
SpeedSketch	в $1.59\times$ быстрее ODESS	кодирование в $1.92\times$ быстрее Gdelta

Без прямых замеров остался только Finesse: в источниках есть лишь его соотношения с другими методами (ODESS быстрее него в $5.4\times$, Palantir лучше него по сжатию на 26.5%), поэтому в таблице он и помечен прочерком.

Из таблицы видно, что в литературе нет единого «лучшего» метода: ODESS выигрывает скоростью, Palantir – качеством, SpeedSketch – скоростью связки «скетч + дельта». Для MarLine это означает, что наличие двух ветвей сходства – ODESS-хешера и иерархических суперпризнаков Palantir – не избыточность, а два варианта решения проблемы. Их вклад возможно оценить только экспериментально.

По разбиению на чанки выбора, по сути, нет: FastCDC 2020 – практический стандарт, и его использование в MarLine соответствует литературе. По дельта-кодированию Gdelta – устойчивый компромисс между скоростью и качеством, и его выбор также обоснован.

7. Выводы

Из обзора следует три вывода.

Во-первых, архитектура MarLine повторяет проверенные решения: разбиение – FastCDC 2020, сходство – ODESS и Palantir, дельта – Gdelta, платформа – ChunkFS. Выбор обоснован литературой, менять его на этом этапе работы не нужно.

Во-вторых, новизна работы не в отдельном алгоритме, а в интеграции и проверке: в одной системе сосуществуют две ветви дедупликации на общей платформе, и ни одна рассмотренная статья не даёт ни детерминированных замеров, ни интеграционных тестов, падающих при ухудшении сжатия. Заполнение этого пробела – и есть предрелизная подготовка.

В-третьих, из анализа следует необходимость сравнить ветви ODESS и Palantir экспериментально: литература даёт для каждой свои цифры, но на разных наборах данных, и перенос итогов на MarLine без собственного замера некорректен.

8. Следствия для постановки задачи

Корректировка цели и задач практики по результатам обзора не требуется. Сформулированная постановка – интеграционное тестирование обеих ветвей, устранение дефектов, проверка качества сжатия, приведение документации и CI к публикуемому виду – уже включает всё, что обзор показывает как пробел в литературе. Таким образом, результаты обзора подтверждают постановку, а не заставляют её изменять.

Список литературы

[1] W. Xia, Y. Zhou, H. Jiang, D. Feng, Y. Hua, Y. Hu, Y. Zhang, Q. Liu. *FastCDC: a Fast and Efficient Content-Defined Chunking Approach for Data Deduplication*. Proceedings of USENIX Annual Technical Conference (ATC), Denver, CO, 2016, pp. 101–114.

[2] W. Xia, X. Zou, H. Jiang, Y. Zhou, C. Liu, D. Feng, Y. Hua, Y. Hu, Y. Zhang. *The Design of Fast Content-Defined Chunking for Data Deduplication Based Storage Systems*. IEEE Transactions on Parallel and Distributed Systems, vol. 31 (9), 2020, pp. 2017–2031.

[3] L. Aronovich, R. Asher, E. Bachmat, H. Bitner, M. Hirsch, S.T. Klein. *The Design of a Similarity Based Deduplication System*. Proceedings of SYSTOR 2009: The Israeli Experimental Systems Conference, Haifa, 2009, pp. 1–14.

[4] X. Zou, C. Deng, W. Xia, P. Shilane, H. Tan, H. Zhang, X. Wang. *Odess: Speeding up Resemblance Detection for Redundancy Elimination by Fast Content-Defined Sampling*. 2021 IEEE 37th International Conference on Data Engineering (ICDE), Chania, 2021, pp. 480–491.

[5] H. Huang, P. Wang, Q. Su, H. Xu, C.J. Xue, A. Brinkmann. *Palantir: Hierarchical Similarity Detection for Post-Deduplication Delta Compression*. Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), vol. 2, La Jolla, 2024, pp. 830–845.

[6] F. Yang, Y. Wang, C. Li, K. Zhou, H. Li. *SpeedSketch: An Ultra-Fast Sketch Generation and Delta Encoding Framework for Delta Compression*. Proceedings of the 54th International Conference on Parallel Processing (ICPP), San Diego, 2025, pp. 168–177.

[7] H. Tan, Z. Zhang, X. Zou, Q. Liao, W. Xia. *Exploring the Potential of Fast Delta Encoding: Marching to a Higher Compression Ratio*. 2020 IEEE International Conference on Cluster Computing (CLUSTER), Kobe, 2020, pp. 198–208.

[8] O. Piletskii. *ChunkFS* – файловая система для сравнения алгоритмов дедупликации. `https://github.com/Piletskii-Oleg/chunkfs`